



# Semana 06: Relaciones en SQLAlchemy + Service Layer



## Objetivos de Aprendizaje

Al finalizar esta semana, serás capaz de:

- ☒ Implementar relaciones uno a muchos (1:N) entre modelos
- ☒ Implementar relaciones muchos a muchos (N:M) con tablas asociativas
- ☒ Realizar queries eficientes con joins y eager/lazy loading
- ☒ Entender el patrón Service Layer y su propósito
- ☒ Separar lógica de negocio de los endpoints (routers)
- ☒ Estructurar proyectos FastAPI con capas de responsabilidad



## Contenidos

### 1. Teoría

| #  | Tema                             | Archivo                                          |
|----|----------------------------------|--------------------------------------------------|
| 01 | Relaciones Uno a Muchos (1:N)    | <a href="#">01-relaciones-uno-a-muchos.md</a>    |
| 02 | Relaciones Muchos a Muchos (N:M) | <a href="#">02-relaciones-muchos-a-muchos.md</a> |
| 03 | Queries con Relaciones           | <a href="#">03-queries-con-relaciones.md</a>     |
| 04 | Introducción al Service Layer    | <a href="#">04-introduccion-service-layer.md</a> |
| 05 | Implementando Servicios          | <a href="#">05-implementando-servicios.md</a>    |

### 2. Prácticas

| #  | Ejercicio                                     | Duración |
|----|-----------------------------------------------|----------|
| 01 | <a href="#">Relación 1:N - Author → Posts</a> | 40 min   |
| 02 | <a href="#">Relación N:M - Posts ↔ Tags</a>   | 40 min   |
| 03 | <a href="#">Queries con Relaciones</a>        | 35 min   |
| 04 | <a href="#">Refactorizar a Services</a>       | 45 min   |

### 3. Proyecto

**Blog API con Service Layer** - API completa para un blog con:

- Authors, Posts y Tags
- Relaciones 1:N y N:M
- Arquitectura de capas (Routers → Services → Models)

 [Ver proyecto](#)



## Arquitectura de Esta Semana

Esta semana introducimos el **Service Layer**:

ANTES (Week-05)

```
Router/Endpoint
├─ Validación (Pydantic)
├─ Lógica de negocio      ← Todo mezclado
├─ Acceso a DB
└─ Response
```

#### AHORA (Week-06)

```
Router/Endpoint
├─ Validación (Pydantic)
└─ Llama a Service
    ↓
Service
├─ Lógica de negocio      ← Separado
└─ Acceso a DB
```

## Estructura de archivos

```
src/
├─ main.py                # FastAPI app
├─ database.py            # Engine, Session
├─ routers/               # ← Endpoints HTTP
│   ├── __init__.py
│   ├── authors.py
│   └─ posts.py
├─ services/              # ← NUEVO: Lógica de negocio
│   ├── __init__.py
│   ├── author_service.py
│   └─ post_service.py
├─ models/                # SQLAlchemy Models
│   ├── __init__.py
│   ├── author.py
│   └─ post.py
└─ schemas/               # Pydantic Schemas
    ├── __init__.py
    ├── author.py
    └─ post.py
```



## Distribución del Tiempo

| Actividad                | Tiempo         |
|--------------------------|----------------|
| Teoría (5 temas)         | 2 horas        |
| Prácticas (4 ejercicios) | 2.5 horas      |
| Proyecto                 | 1.5 horas      |
| <b>Total</b>             | <b>6 horas</b> |



## Requisitos Previos

- ☒ Week-05: SQLAlchemy ORM básico
- ☒ CRUD completo con FastAPI
- ☒ Pydantic schemas
- ☒ Dependency Injection ( Depends )

---

## Entregable

Proyecto: [Blog API](#)

API de blog funcionando con:

- ☐ Relaciones 1:N y N:M implementadas
- ☐ Service Layer para lógica de negocio
- ☐ Endpoints testeados con Swagger

---

## Navegación

| ← Anterior                                | Actual           | Siguiente →                                   |
|-------------------------------------------|------------------|-----------------------------------------------|
| <a href="#">Semana 05: SQLAlchemy ORM</a> | <b>Semana 06</b> | <a href="#">Semana 07: Repository Pattern</a> |

---

## Recursos

- [SQLAlchemy Relationships](#)
- [FastAPI Bigger Applications](#)
- [Service Layer Pattern](#)